BACKEND DEVELOPER

PORTFOLIO

사용자의 불편을 발견하고
기술로 개선하는 개발자

업무 시스템 개발·운영 1년 3개월

Java Spring Boot AWS



김영만

github.com/sksn12

sksn12@naver.com

PROFILE

경력 및 역량

2026.04 - 2026.09 — 교육

KOSA 현대퓨처넷 11기

MSA 기반 Full Stack 개발 과정

2024.01–2025.04 — 경력 · 1년 3개월

이도

IT사업팀 / 업무 시스템 개발·운영

2022.07–2023.06 — 교육

SSAFY

삼성 청년 SW 아카데미

2016.03–2022.08 — 학력

인천대학교

정보통신공학과 / 졸업

수상

KOSA 최종 프로젝트 우수상

2026.09.04 · StockFit · 팀 수상

SSAFY 공통 프로젝트 우수상

2023.02.17 · 삼성 청년 SW 아카데미

자격 및 어학

01 **정보처리기사**

02 **SQLD**

03 **리눅스마스터 2급**

04 **TOEIC Speaking IM3**

WORK EXPERIENCE

사내 업무 시스템 개발과 운영 개선

현업 요구사항을 기능으로 구현하고, 배포 이후 발견한 문제까지 개선했습니다.

담당 업무

현업 요구사항 분석 및 화면·API 개발

사용자의 실제 업무 절차를 확인하고 필요한 기능을 구현

환경사업소 시스템과 올바로 EDI 연동

내부 인계·처리 정보를 올바로 전자인계서로 자동 전송

DB와 웹 화면의 성능 개선

실행 계획과 사용자 지표로 병목을 확인한 뒤 구조를 개선

배포와 서비스 운영

Tomcat·Nginx 배포, Jenkins 자동화, 로그 점검과 장애 대응

대표 성과

2초 이상에서 0.1초 미만으로

150만 건 통계 API 응답 시간 약 95% 단축

3.5초에서 2.1초로

사진 목록 LCP 개선, 사용자 이탈률 약 15% 감소

API Key 후보 2건 추가 발견

자동 탐지 도구로 기존 점검에서 누락된 후보 확인

WORK EXPERIENCE

환경사업소 업무 시스템 공통화와 올바로 EDI 연동

폐기물 반입부터 계약·계근·처리 현황까지 관리하는 시스템을 공통화하고, 행정 신고까지 자동으로 연결했습니다.

프로젝트 배경

어떤 시스템인가

폐기물을 반입한 거래처의 계약 정보와
계근 결과, 처리 현황을 관리하는 내부 업무 시스템

기존 문제

사업소마다 업무 절차와 기존 시스템이 달라
신규 사업소마다 별도로 개발·운영

전체 업무 흐름

폐기물 반입 → 계근 → 계약 확인 → 처리 현황 관리 → 행정 신고

프로젝트 목표

사업소별 업무 차이를 반영한 공통 시스템 전환과 전자인계서 자동 전송

행정 신고 절차의 변화

BEFORE

담당자 → 직접 입력 → 내부 업무 시스템
폐기물 인계·처리 정보 입력

담당자 → 다시 입력 → 올바로시스템
같은 정보를 전자인계서에 재입력

수동 입력 2회

↓

AFTER

담당자 → 내부 업무 시스템
인계·처리 정보 1회 입력 → EDI 자동 전송 → 올바로시스템
한국환경공단 행정 신고

담당 역할 및 결과 현장 업무 확인 · 거래처·계약·처리 현황 화면과 API 개발 · 올바로 EDI 연동 / 반복 입력 절차 감소

WORK EXPERIENCE

150만 건 Full Scan을 제거해 평균 응답을 0.1초 미만으로 단축

Pinpoint와 실행 계획으로 병목을 확인하고, 복합 인덱스와 쿼리 단순화 후 같은 조건으로 검증했습니다.

BEFORE

2.0초 이상

AFTER

0.1초 미만

RESULT

약 95% 개선

통계 조회 대기 시간을 줄여 결과를 즉시 확인할 수 있게 했습니다.

01 TROUBLESHOOTING

01 문제 출입 통계 API가 평균 2초 이상 걸렸고, 데이터가 늘수록 지연이 심해졌습니다.

02 원인 Pinpoint와 실행 계획에서 약 150만 건 로그 테이블의 Full Scan과 불필요한 JOIN을 확인했습니다.

03 개선 학교 ID, 날짜, 학생 ID 조건과 B-Tree 특성을 고려해 복합 인덱스를 설계하고 쿼리를 단순화했습니다.

분석 결과를 근거로 변경 전후를 같은 조건에서 검증했습니다.

02 ANALYSIS RECONSTRUCTION

당시 분석 내용 재구성

BEFORE

150만 건 로그 전체 조회

Full Scan · 불필요한 JOIN

2초 이상

AFTER

복합 인덱스 적용

조건 기반 Index Range Scan

0.1초 미만

KOSA FINAL PROJECT · STOCKFIT

최종 프로젝트 우수상

StockFit 통합 AI 재고 관리 플랫폼

판매 채널별로 분산된 재고를 통합하고, 수요 예측과 소비기한 분석으로 위험 재고를 조기에 발견해 대응 방안까지 제시하는 플랫폼입니다.

01 기존 업무의 문제

> 판매 채널별 **재고가 분산돼** 담당자가 여러 시스템을 오가며
> 현황을 확인해야 했습니다.
> 현재 재고 수량만으로 품절·폐기 가능성을 판단해야 해
> **위험 재고에 선제적으로 대응하기 어려웠**습니다.

02 StockFit의 해결 방식

> 채널별 재고를 통합하고, 머신러닝 수요 예측과 소비기한을
> 바탕으로 품절·폐기 위험 재고를 조기에 식별했습니다.
> **생성형 AI가 할인·재고 이동 등 대응 방안과 예상 효과를 제시**하고,
> 실행 전후의 성과까지 확인할 수 있도록 구성했습니다.



담당 영역

팀장 · 대시보드/통계 · 인프라 구축

프로젝트 주요 기술

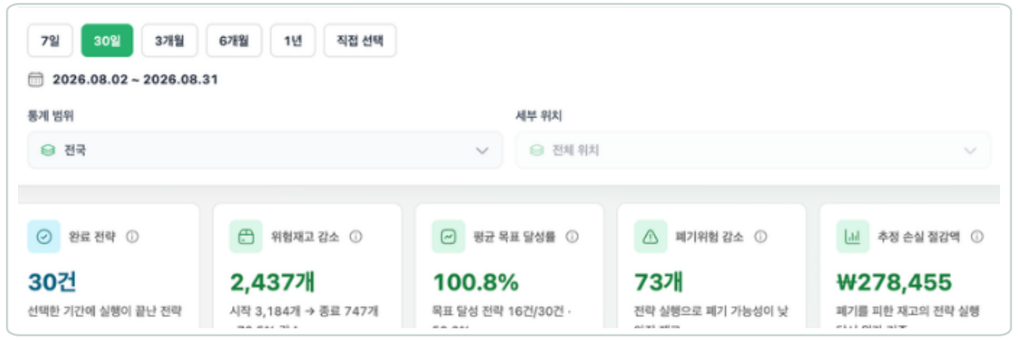
BACKEND	Java 17 · Spring Boot · MyBatis · Spring Batch
DATA	Oracle · Redis · RabbitMQ · Cosmos DB
AI / FRONT	React · Vite · Azure ML · LightGBM · FastAPI
INFRA	AWS · Docker · Nginx · Jenkins · GitHub Actions · Prometheus · Grafana · Loki

BACKEND TROUBLESHOOTING

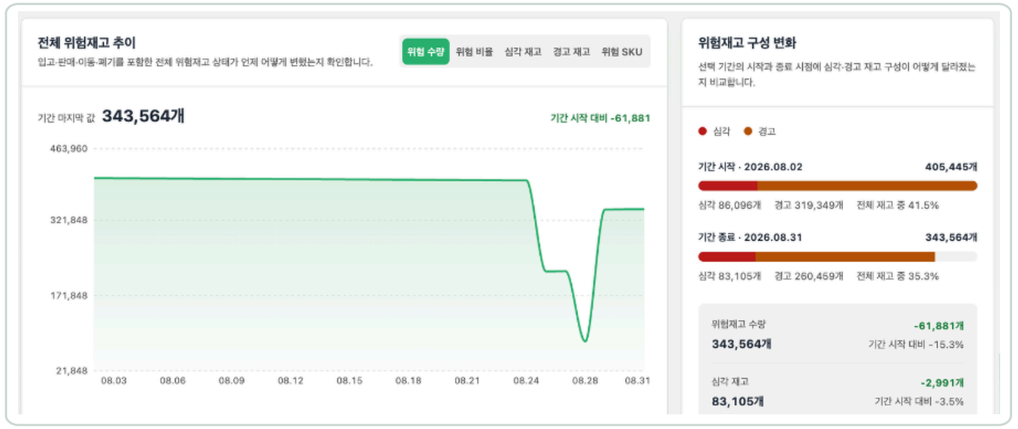
재고 동기화 시 한 번 집계해 반복 계산 제거

재고 동기화가 완료되면 범위별 통계를 JSON 스냅샷으로 저장하고, 화면에서는 최신·기간별 스냅샷을 조회하도록 구조를 바꿨습니다.

AI 전략 성과 요약



위험 재고 추이



BEFORE

조회할 때마다 통계를 다시 계산

화면을 조회할 때마다
재고·LOT·위험등급·수요예측
데이터를 다시 조회하고
통계를 집계했습니다.

PROBLEM

요청마다 복잡한 집계 반복
조회 시점에 따라 기준 시각 변동

AFTER

동기화 시 한 번만 통계를 집계

재고 동기화가 완료되면
범위별 통계를 한 번 집계해
JSON 스냅샷으로 저장하고,
최신·기간별 결과를 조회했습니다.

RESULT

화면 요청 시 복잡한 집계 제거
동일한 기준 시각으로 통계 제공

Infra

StockFit의 AWS 운영 구조

사용자 요청 처리부터 배포 자동화, 장애 감지와 알림까지 AWS 환경에서 연결했습니다.

---

01 서비스 제공

사용자 요청
HTTPS 접속

→

S3 · CloudFront
프론트엔드 배포

→

Nginx · Spring Boot
백엔드 API

→

Oracle · Redis · RabbitMQ
데이터 · 메시지

02 배포 자동화

GitHub Actions
검증 · 이미지 빌드

→

Amazon ECR
컨테이너 이미지 저장

→

Jenkins
배포 파이프라인 실행

→

Blue / Green
무중단 버전 전환

03 운영 관제

Prometheus · Loki
지표 · 로그 수집

→

Grafana
대시보드 · 장애 감지

→

AI
추정 원인 · 대응 분석

→

Microsoft Teams
담당자에게 전달

운영 효과

배포 오류는 운영 반영 전에 차단하고, 장애 발생 시 원인과 대응 방향을 담당자에게 전달합니다.

DEPLOYMENT SAFETY

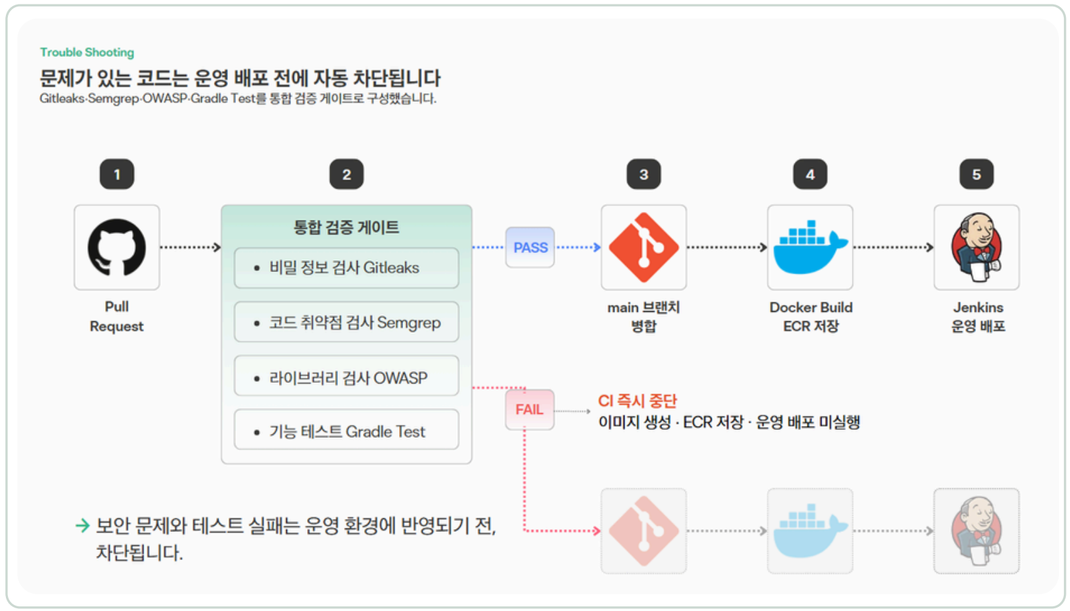
검증된 변경만 배포하고, 정상 버전에만 트래픽을 전환

CI 보안 검사와 테스트를 통과한 이미지만 배포하고, 신규 버전의 Health Check 성공 후 Nginx 트래픽을 전환했습니다.

---

01 PRE-DEPLOYMENT GATE

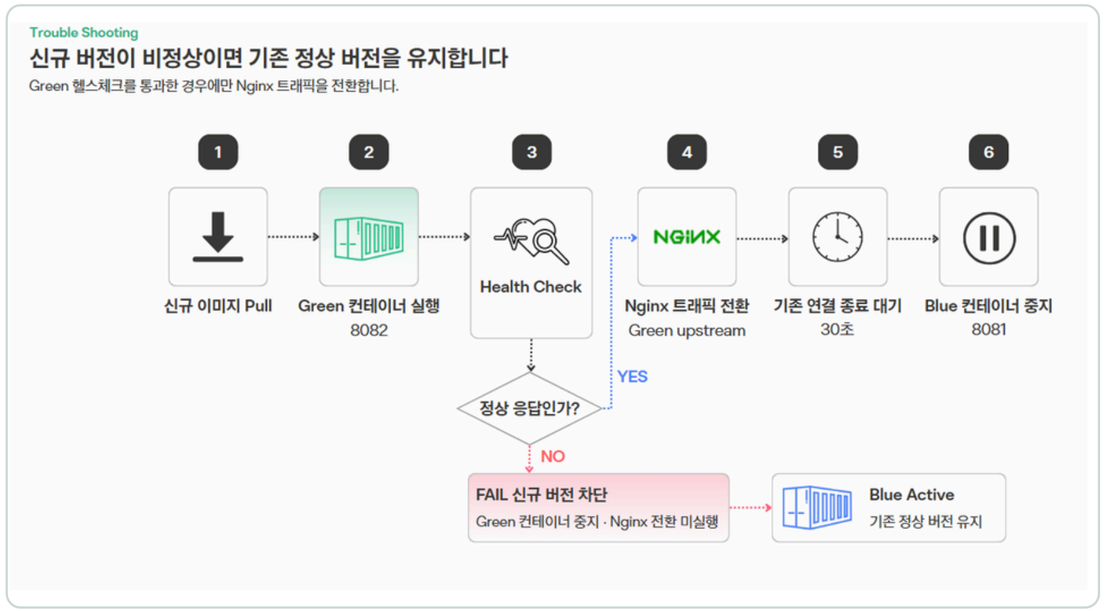
배포 전 검증



검증 항목 중 하나라도 실패하면 이미지 생성과 운영 배포를 중단했습니다.

02 BLUE / GREEN

정상 버전만 전환



Health Check가 실패하면 신규 배포를 중단하고 기존 버전을 유지했습니다.

OPERATIONS TROUBLESHOOTING

AI로 장애 추정 원인과 대응방안을 IT 담당자에게 Teams로 전달

장애 시점의 서버 지표와 오류 로그에서 민감정보를 제거해 AI에 전달하고, 담당자가 원본을 확인한 뒤 최종 조치를 결정하도록 설계했습니다.

01 BEFORE / AFTER

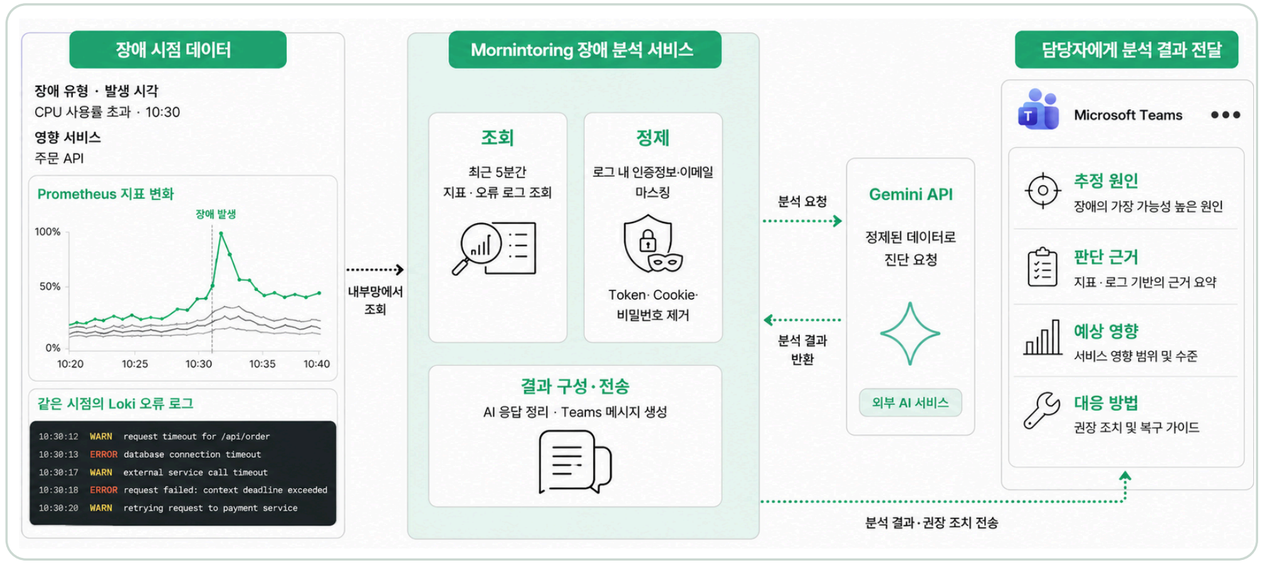
담당자가 받는 경고 메시지



기존에는 장애 사실만, 개선 후에는 원인·근거·영향·조치까지 전달합니다.

02 ANALYSIS FLOW

장애 근거를 분석 결과로 연결



핵심 구현

01 최근 5분의 지표와 오류 로그를 자동으로 수집
02 민감정보를 마스킹한 뒤 AI가 추정 원인과 대응 방법을 정리
03 IT담당자에게 원인과 대응 방안 Teams로 전달

KOSA 2ND PROJECT · SMART MESSAGING

Smart Messaging AI 통합 캠페인 관리 플랫폼

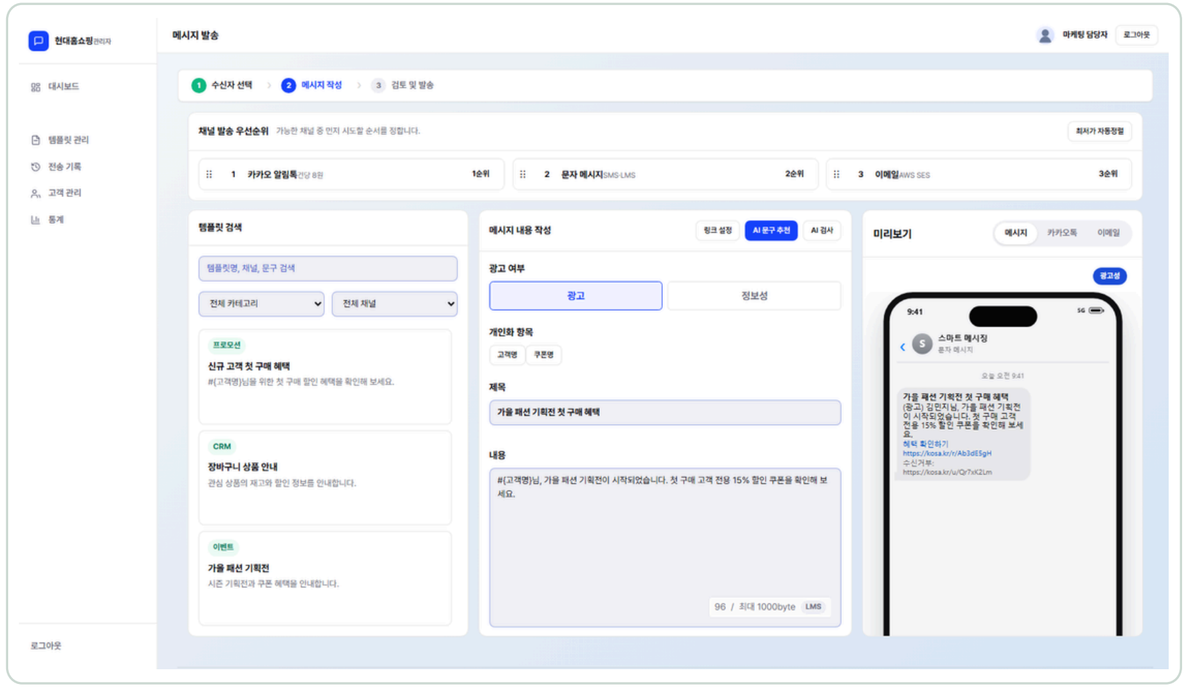
AI 문구 추천부터 수신자 선정, 다채널 대량 발송과 결과 분석까지 하나의 흐름으로 관리합니다.

01 기존 업무의 문제

담당자마다 문구 작성 방식이 달라 브랜드 톤을 유지하기 어려웠고,
수신자 선정·채널 선택·발송·성과 확인 업무도
여러 단계로 분리되어 있었습니다.

02 Smart Messaging의 해결 방식

AI 문구 추천·검사로 기업 광고 메시지 작성을 지원하고,
Redis·RabbitMQ로 대량 발송을 비동기로 처리했습니다.
발송 결과와 비용은 통계 화면에 연결했습니다.



담당 영역

백엔드·DB 설계 · 대량 발송

프로젝트 주요 기술

Java 17 · Spring Boot · Spring Security · Spring Batch · Oracle · MyBatis
Redis ZSet · RabbitMQ · Thymeleaf · JavaScript · Chart.js
Spring AI · Gemini · Qdrant · OpenAI Moderation · Kakao · SOLAPI · AWS SES

MESSAGING TROUBLESHOOTING

대량 발송 요청과 실제 전송을 분리해 실패 작업까지 추적

요청에는 작업 ID를 즉시 반환하고, 대상자 준비와 채널별 발송을 두 RabbitMQ Queue로 나눠 처리했습니다.

01 REQUEST

발송 요청 접수

- **발송 조건과 채널 저장**
  Backend
- **대량 발송 요청 대기열**
  RabbitMQ · 명령 1건

큐 등록 직후 HTTP 응답

02 EXPAND

대량 발송 작업 생성

- **대량 요청 처리 시작**
  Bulk Consumer
- **선택한 고객 목록 조회**
  Redis ZSet
- **연락처와 수신 동의 조회**
  Oracle
- **발송 대상 500명씩 저장**
  Batch Insert
- **고객별 메시지 대기열**
  RabbitMQ

03 DELIVERY

메시지 발송과 결과 기록

- **실제 발송과 테스트 구분**
  Message Consumer
- **알림톡·SMS·Email 발송**
  채널 API
- **시도 결과와 상태 기록**
  send_attempt
- **실패 메시지 보관·재처리**
  대체 채널 · DLQ

01 빠른 요청 처리

발송 완료를 기다리지 않고 응답

02 대량 데이터 처리

Redis 조회와 500건 Batch Insert

03 실패 영향 제한

대체 채널 전환과 DLQ 격리